

Software Testing Tool Study

312553027

Introduction

Jest 是一套由 Facebook 開發的 JavaScript 測試框架，具有簡潔易用的語法、內建 Mock 支援與豐富的測試報告功能，廣泛應用於前端與 Node.js 專案中。

接下來將介紹如何安裝 Jest、撰寫基本單元測試、使用 Mock 函式以及產生測試覆蓋率報告，並透過實際執行結果進行簡單分析。

Implementation

1. 安裝 Jest

在開始使用 Jest 前，先初始化一個 Node.js 專案（如果還沒有 package.json 的話）：

```
1 | npm init -y
```

安裝 Jest：

```
1 | npm install --save-dev jest
```

安裝完成後，在 package.json 中加入測試腳本：

```
"scripts": {  
  "test": "jest"  
}
```

之後即可透過下列指令執行測試：

```
1 | npm test
```

2. 撰寫 unittest

建立一個簡單的函式 `sum.js`：

```
1 | function sum(a, b) {  
2 |   return a + b;  
3 | }  
4 | module.exports = sum;
```

對應的測試檔案 `sum.test.js`：

```
1 | const sum = require('./sum');
2 |
3 | test('adds 1 + 2 to equal 3', () => {
4 |   expect(sum(1, 2)).toBe(3);
5 | });
```

這裡的 `expect` 是 Jest 提供的斷言函式，用來檢查實際輸出結果是否符合預期。搭配 `.toBe()` 等 `matcher` 使用，可以精準比對數值是否相等。舉例來說，`expect(sum(1, 2)).toBe(3)` 表示期望 `sum(1, 2)` 回傳的值應該等於 `3`，若不相等則該測試會失敗，幫助我們快速發現邏輯錯誤。

執行測試：

```
1 | npm test
```

執行測試後，Jest 會清楚列出每個測試案例的通過狀態與測試檔案，讓開發者能快速掌握邏輯正確性。透過這種方式撰寫單元測試，可以有效避免程式邏輯錯誤，提高程式碼的可維護性與穩定性。

以下為測試結果：

```
1 | > tool_study@1.0.0 test
2 | > jest
3 |
4 | PASS ./sum.test.js
5 |   ✓ adds 1 + 2 to equal 3 (2 ms)
6 |
7 | Test Suites: 1 passed, 1 total
8 | Tests:      1 passed, 1 total
9 | Snapshots:  0 total
10 | Time:       0.254 s
11 | Ran all test suites.
```

由測試結果可見所有測試套件（Test Suites）與測試案例（Tests）皆成功通過，顯示目前實作的函式邏輯正確無誤。後續可以進一步擴充測試案例，或引入更多模組進行測試覆蓋。

3. Mock

Jest 內建支援 Mock，可以用於模擬依賴函式。

實作如下：

- 結構

```
1 | mock-example/
2 | ├── fetchData.js           // 被測試邏輯依賴的資料獲取函式
3 | ├── getUserData.js        // 真正要測試的邏輯
4 | ├── getUserData.test.js   // 單元測試與 Mock 設定
5 | └── package.json          // 設定測試腳本
```

- `fetchData.js` - 模擬一個外部 API 請求函式

此函式模擬實際情境中會向外部 API 請求資料的情況。在單元測試中，我們不希望真的去發出網路請求，因此會使用 Mock 來模擬這個函式的回傳行為。

```
1 // fetchData.js
2 async function fetchData(userId) {
3   const response = await fetch(`https://api.example.com/users/${userId}`);
4   return response.json();
5 }
6
7 module.exports = fetchData;
```

- `getUserData.js` - 依賴 `fetchData` 的邏輯主體

這個函式是我們的測試目標，它依賴 `fetchData` 取得資料後再進行處理。為了測試它是否正確處理回傳資料，我們將在測試中 mock 掉 `fetchData`。

```
1 // getUserData.js
2 const fetchData = require('./fetchData');
3
4 async function getUserData(userId) {
5   const data = await fetchData(userId);
6   return `User: ${data.name}`;
7 }
8
9 module.exports = getUserData;
```

- `getUserData.test.js` - 使用 Jest Mock 的單元測試

- 使用 `jest.mock()` 對外部模組進行模擬，不會執行實作內容。
- 使用 `mockResolvedValue()` 直接指定模擬資料回傳值。
- `toHaveBeenCalled()` 可確認 Mock 函式的呼叫參數是否正確。

```

1  // getUserData.test.js
2  const getUserData = require('./getUserData');
3  const fetchData = require('./fetchData');
4
5  // 對 fetchData 進行 mock
6  jest.mock('./fetchData');
7
8  test('should return formatted user name', async () => {
9    // 模擬 fetchData 回傳一筆假資料
10    fetchData.mockResolvedValue({ name: 'Alice' });
11
12    const result = await getUserData(123);
13
14    // 驗證主函式是否正確處理資料
15    expect(result).toBe('User: Alice');
16
17    // 驗證 fetchData 是否正確被呼叫
18    expect(fetchData).toHaveBeenCalledWith(123);
19  });
20

```

• 執行結果

輸出結果如下：

```

1  > mock-example@1.0.0 test
2  > jest
3
4  PASS ./getUserData.test.js
5    ✓ should return formatted user name (5 ms)
6
7  Test Suites: 1 passed, 1 total
8  Tests:      1 passed, 1 total
9  Snapshots:  0 total
10 Time:        0.19 s, estimated 1 s
11 Ran all test suites.

```

測試成功通過表示 `getUserData()` 能夠正確處理來自 `fetchData()` 的資料。

透過 Jest 的 Mock 功能，我們有效隔離了外部依賴，讓測試能專注在內部邏輯的驗證上，進而提升測試的穩定性與重現性，特別適用於需要依賴 API、資料庫或第三方模組的情境。

4. Coverage

以 lab1 為例，在完成 `main.test.js` 中對 `MyClass` 和 `Student` 類別的測試案例後，透過執行下列指令來產生覆蓋率報告：

```

1  npx jest --coverage

```

執行後可得到如下覆蓋率報告摘要：

```

1 PASS ./main.test.js
2   ✓ Test MyClass's addStudent (2 ms)
3   ✓ Test MyClass's getStudentById
4   ✓ Test Student's setName (1 ms)
5   ✓ Test Student's getName
6
7 -----|-----|-----|-----|-----|-----
8 File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
9 -----|-----|-----|-----|-----|-----
10 All files |    100 |    100 |    100 |    100 |
11  main.js  |    100 |    100 |    100 |    100 |
12 -----|-----|-----|-----|-----|-----
13 Test Suites: 1 passed, 1 total
14 Tests:      4 passed, 4 total
15 Snapshots:  0 total
16 Time:       0.371 s, estimated 1 s
17 Ran all test suites.

```

說明：

- **Statements** 100%：所有語句皆已執行過，例如 `push`，`return`，`if` 判斷等。
- **Branches** 100%：所有條件判斷（如 `if/else`）都有被涵蓋，例如 `addStudent` 中對 `instanceof` 的檢查與 `getStudentById` 中的範圍檢查。
- **Functions** 100%：所有定義的函式皆有被呼叫與測試。
- **Lines** 100%：所有實際程式碼行都已執行。

若原本程式碼寫得不夠完備，將導致覆蓋率下降：

以下提供一些假設情境，說明如果原始程式碼寫得不夠健全或測試不足，會如何影響測試覆蓋率：

- 方法定義但未使用（未被呼叫）

如果程式碼裡有某些方法是定義了但沒被測試到，例如在 `Student Class` 中加入 `unusedMethod()`：

```

1 // main.js
2 class Student {
3     constructor() {
4         this.name = undefined;
5     }
6
7     setName(userName) {
8         if (typeof userName !== 'string') {
9             return;
10        }
11        this.name = userName;
12    }
13
14    getName() {
15        if (this.name === undefined) {
16            return '';
17        }
18        return this.name;
19    }
20
21    // 加入未被呼叫的 function
22    unusedMethod() {
23        console.log("This method is never used.");
24    }
25
26 }

```

得到如下覆蓋率報告摘要：

```

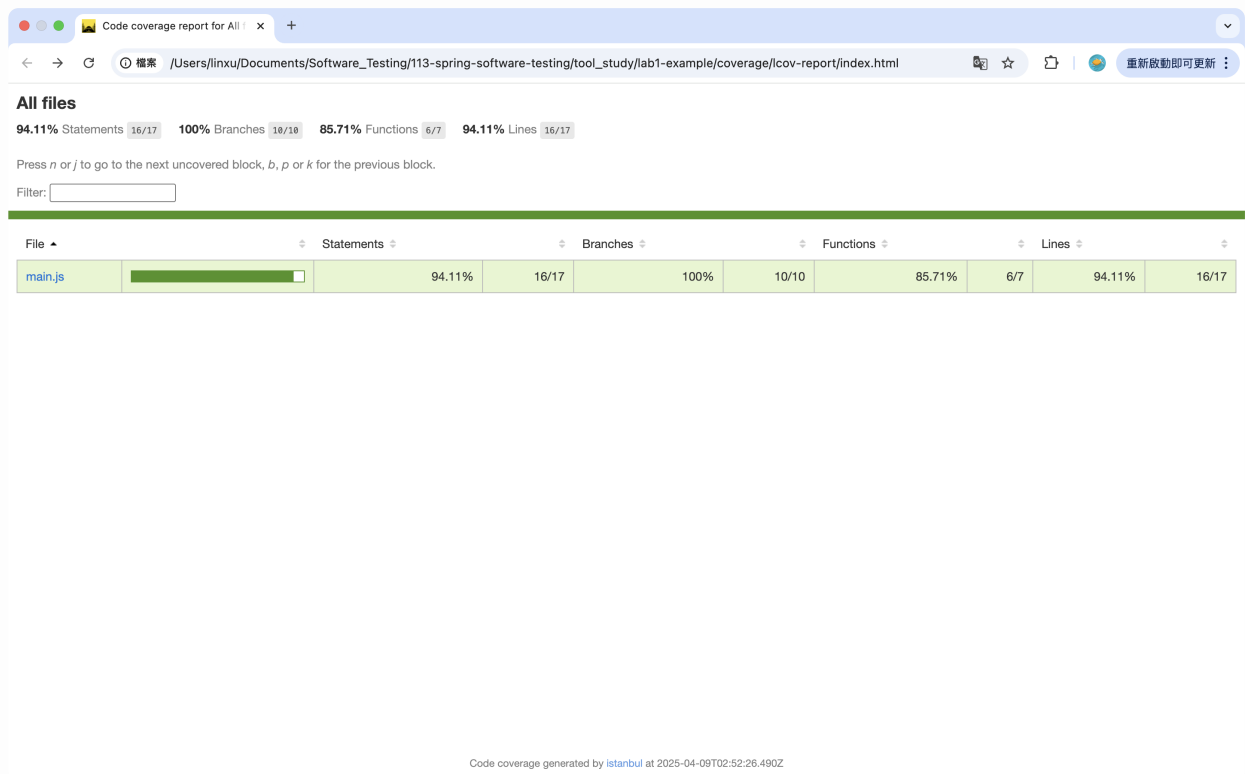
1 PASS ./main.test.js
2   ✓ Test MyClass's addStudent (2 ms)
3   ✓ Test MyClass's getStudentById
4   ✓ Test Student's setName
5   ✓ Test Student's getName
6
7 -----|-----|-----|-----|-----|-----
8 File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
9 -----|-----|-----|-----|-----|-----
10 All files |   94.11 |    100   |   85.71 |   94.11 |
11 main.js   |   94.11 |    100   |   85.71 |   94.11 | 45
12 -----|-----|-----|-----|-----|-----
13 Test Suites: 1 passed, 1 total
14 Tests:       4 passed, 4 total
15 Snapshots:   0 total
16 Time:        0.346 s, estimated 1 s
17 Ran all test suites.

```

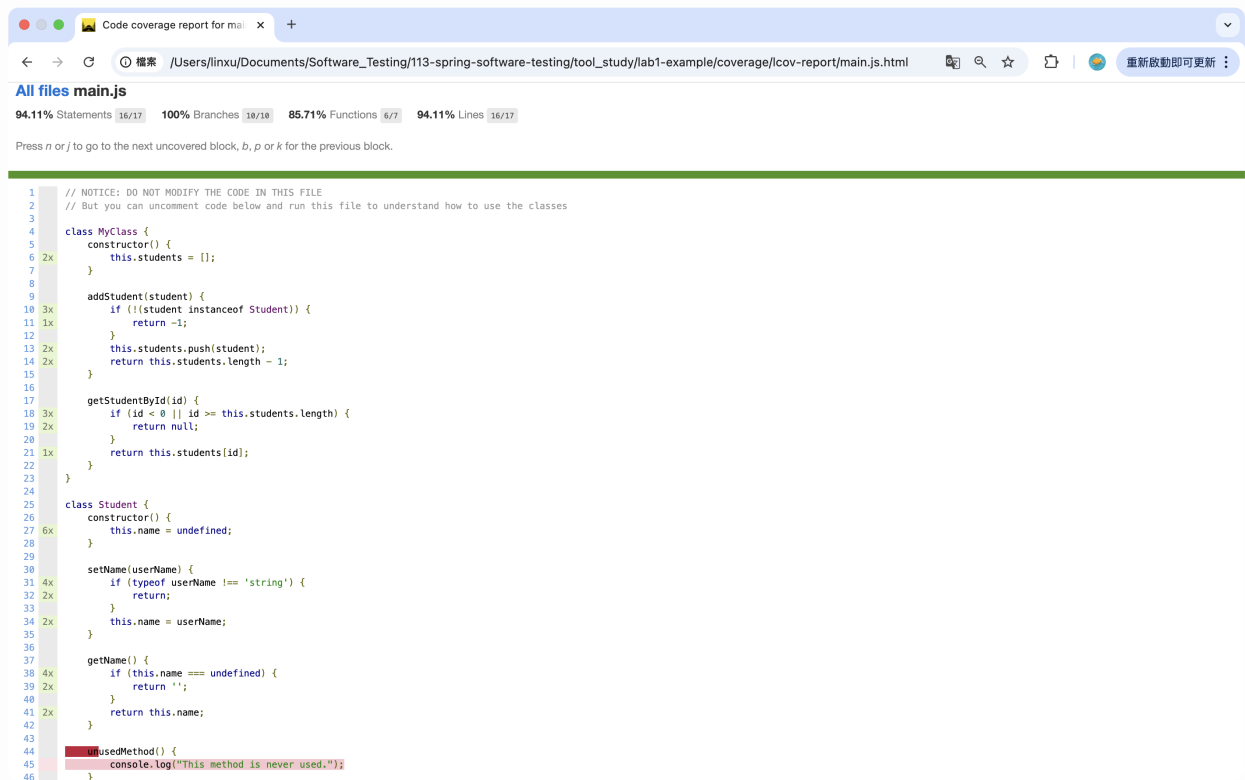
即使測試中涵蓋了大部分功能，只要這個方法沒被呼叫，Functions Coverage 就不會是 100%。

HTML 報表

Jest 也提供測試覆蓋率的圖形化 HTML 報表，執行後會產生 `coverage/lcov-report/index.html`，用瀏覽器打開即可看到測試覆蓋的整體情形：



點選 `main.js` 可查看詳細覆蓋情形



在圖中可以清楚看到哪些行程式碼未被測試覆蓋（紅底），例如某些 if 條件內的錯誤處理、無效參數邏輯、例外狀況等。這讓開發者能精準補測漏掉的邏輯，提升整體測試品質。

Reflections

1. Personal thoughts

使用 Jest 進行 `JavaScript/Node.js` 的單元測試體驗非常直觀。Jest 的語法簡潔，結合內建的測試框架與斷言工具，不需要額外安裝其他第三方 library，對初學者非常友善。整體而言，它降低了測試的門檻，也讓我更願意主動為程式碼撰寫測試案例。

2. Strengths and Weaknesses

優點：

- **開箱即用**：不需要複雜設定就可以直接跑測試。
- **執行速度快**：Jest 具備快取機制與並行執行機制，能快速完成測試。
- **支援 Mock 功能強大**：方便模擬函式或模組，尤其在測 API、DB 時特別有幫助。
- **內建 Coverage 報表**：提供完整的 HTML 覆蓋率視覺化介面，有助於檢查 uncovered 區塊。

缺點：

- **大型專案下可能設定複雜**：如果專案架構較大，或要支援 TypeScript、Vue、React 等框架，設定檔會變得冗長。
- **執行效能偶爾波動**：尤其測試量很大時，有些情況下需要調整執行參數或用 `--runInBand` 模式。
- **對於非 JS 環境支援有限**：像是 Python/C++ 等語言無法使用。

3. Suggestions for improvement

- 若能提供更直覺的 GUI 介面整合（像 Vitest 的報表 UI 或者跟 VS Code 更緊密結合）會更吸引初學者。
- 增加測試案例「自動產生建議」功能，例如根據函式結構幫助初學者建立基本測試架構。
- 在大型 monorepo 下能提供更好的模組相依追蹤與測試隔離會更方便。

4. Comparison with other tools

工具	優勢	缺點
Jest	快速、易用、Mock 功能強、社群活躍	對複雜專案設定可能繁瑣
Mocha	更彈性，適合組合自己想要的斷言與 mock 工具	缺少內建 mock、需要更多設定
Vitest	更快、更接近 Vite、支援 TS 和 ESM 更好	相對較新，資源與支援比不上 Jest
AVA	並行執行快、語法簡潔	缺乏內建 coverage、mock 功能較弱

我個人偏好使用 **Jest**，因為它的**整合性與穩定性高**，即使剛接觸測試也能很快上手，對中小型 Node.js 專案來說特別合適。

4. Will you adopt this tool in the future?

我會在未來的 JavaScript/Node.js 專案中繼續使用 Jest。原因如下：

- 文件齊全，學習曲線低
- 內建 Coverage 支援完整
- 與 CI/CD 工具整合（如 GitHub Actions、GitLab CI）也相當順暢

除非專案需要極高的性能或特殊前端環境，我才會考慮改用 Vitest 或其他工具。